

My intentions for the AE3 project was to move past some of the conventional graph theory that we had experienced in the lectures and build an application that wouldn't crash during a rainy day or exception path. Throughout my semester here, my programming perspective has shifted away from just satisfying all the outputs and simply "working" to trying to account for unexpected outcomes in object oriented systems.

Instead of solely relying on the offline dataset that was given, I was able to integrate the live WebTRIS API client that I developed in AE 2. This decision actually introduced some of the real-world problems associated with exception handling as by using the live API, sensors break or are offline, and API rate limitations create a threshold for connections. Some of the issues I encountered were the live API frequently dropping data in between junctions 7 to 12. Rather than letting the edge weight calculation logic to crash with ZeroDivisionError, I tried to design a system to help catch those exceptions, log None, and use a 40.0 mph speed if the whole section goes down. To complement this, I also made a purposeful design decision to split the large 23 mile distance in between junctions 7-12 into five 4.6 mile sections. While not rigidly required by the math used, I did make the decision to make sure that my algorithms like BFS would correctly count the actual junction stops.

To totally analyze the M25 road system in between junction 7 (Gatwick Airport) and Heathrow Airport, I implemented the 3 major search algorithms of BFS, DFS, and Dijkstra's to optimize for function for their different unique limitations and strengths. As a result of these differences, they for the most part returned different results for the same road route network analyzed.

Breadth First Search (BFS):

The BFS algorithm explores a graph by exploring outward level by level. By utilizing a deque or double ended queue, it can explore the graph basically layer by layer and guarantees finding the shortest path through finding the fewest junction stops/edges that were traversed, but completely ignores travel times or the edge weights that are involved.

During runtime, BFS chose route B and diverted at junction 12 on the M25 road network via M3: Sunbury-on-Thames. The algorithm was able to complete the route to Heathrow Airport in 8 junction stops/nodes with an estimated time of 65.32 minutes.

BFS notoriously ignored the 20 minutes flat time it took by traversing the road stretch from Sunbury, and rather chose this option just because it required fewer "steps" or junctions to reach the target destination. As such BFS is completely blind to the time edge weights, and as a result chose route B just because it required navigation through less junctions than the more direct route A.

From a business/software point of view, BFS is more preferable to other algorithms when there are no edge weights or costs associated with nodes travelled. For example, finding the least amount of connections between suggested users on social-media likely uses BFS in some regard as it relies on finding the least amount of connections between one user and another.

This can have the effect of creating the highly personalized connection suggestion feature on social media platforms such as Instagram or Facebook. Other applications of BFS include the logistical transfer of freight shipping as well as routing network packets with least router hops are many real world applications that can be implemented using the BFS algorithm.

Dijkstra's Algorithm:

Using the min-heap or a priority queue, Dijkstra's is a greedy algorithm that finds the cheapest weighted known route at every step/move it makes. It is a mathematically guaranteed path that gives the lowest weighted travel time edge.

During the running of the algorithm, Dijkstra's algorithm chose route A, remaining on a direct path on the M25 from junction 13 to 14 with an estimated travel time of 40.45 minutes across the 9 junctions.

Compared to BFS, Dijkstra's algorithm analyzed the actual pulled speed data from the WebTRIS API and calculated that remaining on the high-speed and more monitored M25 road network for more junctions was algorithmically faster than local road diversions in route B. This shows the overall trade-offs between the junction amounts and the speed found in the WebTRIS logic.

Dijkstra's algorithm remains one of the most, if not most important standards for time-sensitive routing ranging from GPS navigation systems like Google Maps, EMS/ambulance vehicle routing, or high frequency trade networks. All of these examples rely on finding the lowest latency, regardless of how many steps or junctions are involved.

Depth First Search (DFS):

DFS works by completing a recursive dive into the first branch it runs into up until it reaches the target destination or hits a dead end which leads to the backtracking feature being activated. It is able to guarantee a valid path to the destination, but it can't guarantee the fastest mathematical path being found in regards to the time constraint weights or junction/nodes taken for that path.

The DFS algorithm selected route A which involved a traversal over 9 junctions in 40.45 minutes.

DFS was able to choose route A solely by the coincidence of the order of the dictionary insertions. Because the junctions for route A were constructed and instantiated into the neighboring list first in the graph building phase, the algorithm blindly followed it directly all the way to the Heathrow target destination. If another diversion such as on the M3 was inserted first instead, DFS would likely have taken that route.

While ineffective for time-restrained GPS navigating, DFS is rather efficient in memory-constrained use cases because it only requires storing the current path taken within the call stack instead of branching out to all nodes such as in BFS. Robotics as well as maze

solving applications implement DFS into their features for when an agent must explore an unfamiliar network without keeping the whole map in the memory.

Testing and Verification:

Due to the fact that the live WebTRIS API data tends to fluctuate every day, it is rather difficult to prove the accuracy of the 3 algorithms in using the live M25 Network. For instance, a route that may be considered most efficient today may not be tomorrow, and in order to verify the accuracy of all 3 algorithms across multiple environments without depending on too many unpredictable datasets, I created a fake mock graph using pytest using 2 different routes to the target. The first was a direct route composed of 1 edge, but was extremely traffic-heavy/congested taking 30 minutes in total. The second was an indirect route that was composed of 2 edges via a midpoint, but was faster taking 10 minutes per segment making up a roughly 20 minute estimated time.

Regarding the testing outcomes, BFS using `test_bfs_algo` was able to verify that the algorithm prioritized taking the least amount of junctions. It ignored the faster 20 minute route to choose the 1 edge direct route, despite taking 30 minutes, which correlates to its core logic of ignoring weighted edges in graph traversal.

The `test_dijkstra` test case verified that min-heap accurately prioritized the lowest some of the edge weights in its chosen route. It ignored the direct route, instead choosing the 2-edge second route that took 20 minutes.

The `test_dfs_algo` test was able to prove that depth-first-search was able to find a working, connected route from the start point to the target. While not able to guarantee the most efficient path of the least number of junctions hit like the other 2 algorithms, the test was able to recursively navigate the junctions and calculate a working travel time without crashing.

Going beyond testing the basic accuracy of the routing algorithms, I also tested `webtris_graph` against broken or incorrect API data. For example, in my `test_edge_getter`, I input the parser arrays that contained None parameter values to act like the broken WebTRIS sensors. I used the test to successfully show that my program filtered out the offline sensors and defaulted to the 40 mph when a whole section of the road sensors went offline, thus preventing a `ZeroDivisionError` from being raised.

```
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\...Downloads\CS_Labs\week-3-lab\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\...Downloads\CS_Labs\webtris_project
collected 6 items

test_webtris_graph.py::test_location_graph PASSED [ 16%]
test_webtris_graph.py::test_edge_time_getter PASSED [ 33%]
test_webtris_graph.py::test_bfs_algo PASSED [ 50%]
test_webtris_graph.py::test_dijkstra PASSED [ 66%]
test_webtris_graph.py::test_dfs_algo PASSED [ 83%]
test_webtris_graph.py::test_unreachable_dest PASSED [100%]

===== 6 passed in 0.08s =====
(week-3-lab) PS C:\Users\...Downloads\CS_Labs\webtris_project>
```

pytest output

In conclusion, my primary intention for this project was to integrate some of the concepts we learned in graph theory and traversal we learned in weeks 8-9. Trying to incorporate the lecture concepts with unpredictable live API data altered my overall outlook with my architecting strategy for my program while also accounting for real-world logistical issues such as broken sensors. Opposed to solely satisfying requirements or spitting out the algorithmic data, I tried to carefully design the entire pipeline for my data and how it flows throughout the entirety of my program. I first mapped out how the JSON response would get filtered by the time calculations, which then gets turned into the Location object, and would then get sorted by the min-heap priority queue.